BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS
ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

3D NEURAL CELLULAR AUTOMATA

3D NEURÁLNÍ CELULÁRNÍ AUTOMATY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **MICHAL REPČÍK**
AUTOR PRÁCE

SUPERVISOR **Ing. KAREL FRITZ,**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v anglickém jazyce.

Abstrakt

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Klíčová slova

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Reference

REPČÍK, Michal. *3D Neural Cellular Automata*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Karel Fritz,

3D Neural Cellular Automata

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Karel Fritz I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used ChatGPT to correct spelling and other language mistakes. I used Copilot when working on the software.

. . . . . .
Michal Repčík
March 2, 2026

Acknowledgements

I would like to express my sincere thanks to my supervisor, Ing. Karel Fritz, for his guidance and support.

Contents

1 Introduction **3**

2 Abstract **4**

3 Theoretical Foundations **5**
3.1 Prerequisites 5
3.2 Cellular Automata 5
3.3 Neural Cellular Automata 7
3.4 Extension from Two to Three Dimensions 7
3.5 Core Components of a 3D Neural Cellular Automaton 8

4 Related Work **11**
4.1 Growing Neural Cellular Automata 11

5 Implementation **15**

6 Testing **16**

7 Experiments **17**

8 Conclusion **18**

Bibliography **19**

List of Figures

Chapter 1

Introduction

TODO

Chapter 2

Abstract

TODO

Chapter 3

Theoretical Foundations

3.1 Prerequisites

This thesis assumes familiarity with fundamental concepts in modern deep learning, in particular convolutional neural networks (CNNs), gradient-based optimisation, and automatic differentiation as implemented in frameworks such as PyTorch [17, 4]. Comprehensive introductions to these topics are provided in Goodfellow et al. [7] and in the official PyTorch documentation [4].

The remainder of the thesis is self-contained with respect to cellular automata (CA) and their neural extensions.

3.2 Cellular Automata

Cellular automata (CA) are discrete dynamical systems composed of a regular grid of cells, each of which occupies one state from a finite set. The grid evolves in parallel over discrete time steps according to a local update rule that depends solely on the states of neighbouring cells [12].

Formally, a cellular automaton is defined by:

- a **lattice** of cells, typically arranged in one, two, or three dimensions,
- a **finite state set** $\mathcal{S}$,
- a **neighbourhood function** $\mathcal{N}$ describing which cells influence a given cell,
- an **update rule** $f : \mathcal{S}^{|\mathcal{N}|} \to \mathcal{S}$ that determines the next state [8].

The foundational work of Ulam and von Neumann in the 1940s established CA as a model for self-replication [15]. Later, Wolfram proposed a widely used behavioural classification of CA into four qualitative classes [22, 23]. Class 1 systems quickly converge to a uniform state. Class 2 systems evolve into static or periodically oscillating patterns. Class 3 systems exhibit chaotic and seemingly unpredictable behaviour. Class 4 systems demonstrate complex dynamics characterised by long-lived structures capable of interaction and computation.

These classes remain a reference point in subsequent work on both classical and neural CA.

3.2.1 One-Dimensional Cellular Automata

The simplest non-trivial CA operate on a one-dimensional array of cells. In the elementary CA family, each cell observes its own state and that of its two immediate neighbours (radius 1, neighbourhood size 3). With binary states, the update rule has $2^3 = 8$ possible neighbourhood configurations, yielding $2^8 = 256$ distinct rules in Wolfram's enumeration scheme [22].

A rule number encodes the next-state lookup table in binary. Notable examples include Rule 90, which generates the well-known Sierpiński triangle from a single active seed (Class 3), and Rule 30, whose chaotic evolution has been used as a source of pseudorandomness. Rule 110 (Class 4) is the smallest known Turing-complete 1D CA, supporting gliders and glider collisions that can perform universal computation [5].

3.2.2 Two-Dimensional Cellular Automata

In 2D, the Moore neighbourhood (a 3×3 block) is the most common. Because the space of lookup tables is large (2^9 input combinations), rules are often expressed using birth/survival notation.

Conway's Game of Life [6] remains the most influential 2D CA, featuring a wide variety of emergent structures, including still lifes (e.g. block, beehive), oscillators (e.g. blinker, toad), spaceships (e.g. glider, lightweight spaceship), and more complex constructions such as glider guns and other computational structures.

Life is Turing-complete [1] and has influenced much of the later work on self-organising neural CA models.

3.2.3 Three-Dimensional Cellular Automata

In three dimensions, the Moore neighbourhood includes up to 26 adjacent cells (face-, edge-, and corner-adjacent) in a $3\times3\times3$ cube. With binary states, this creates $2^{27} \approx 134$ million possible neighbourhood configurations, so practical 3D CA typically use totalistic or outer-totalistic rules that depend only on the sum of live neighbours (not their exact positions) [23].

Notable examples include von Neumann's 29-state self-replicating automaton [15], excitable-media rules such as Brian's Brain extended to 3D [10], and three-dimensional Game of Life variants capable of producing branching and crystal-like growth patterns.

Advances in computing hardware in the 2000s facilitated larger-scale 3D CA studies [21]. These volumetric models later aligned naturally with neural CA research.

3.2.4 Differentiability and Continuous Cellular Automata

Although this thesis uses a discrete 3D lattice, the update rule is fully differentiable, enabling gradient-based optimisation [2]. This connects neural CA to continuous cellular automata, where states are real-valued and update rules are smooth rather than discrete.

SmoothLife [18] generalises the Game of Life to continuous space and time using continuous averaging kernels and smooth transition functions, producing gliders and complex wave-like structures on a continuous field.

Lenia [3] further extends this idea using parameterised convolution kernels and growth functions. Its exploration of parameter space led to hundreds of stable, self-organising patterns resembling an artificial ecology.

These systems demonstrate that complex self-organisation does not depend on discrete state transitions. Instead, strictly local, differentiable interactions are sufficient to generate stable and adaptive spatial structures. Neural cellular automata leverage this property while introducing optimisation through gradient descent, allowing update rules to be learned for specific structural objectives rather than discovered through manual parameter exploration.

3.3 Neural Cellular Automata

Neural Cellular Automata (NCA) were introduced by Mordvintsev et al. [13] as a differentiable generalisation of classical cellular automata. In their formulation, the discrete lookup-table update rule of traditional CA is replaced by a small neural network whose parameters are shared across all cells. While the spatial lattice remains discrete, the state of each cell becomes a real-valued vector, enabling gradient-based optimisation through time.

Let the lattice be denoted by $\Omega \subset \mathbb{Z}^d$, where d represents the spatial dimension. Each cell $x \in \Omega$ maintains a state vector

$$\mathbf{s}_t(x) \in \mathbb{R}^C,$$

where C is the number of channels.

The global configuration at time t can therefore be viewed as a tensor containing the states of all cells.

Following Mordvintsev et al. [13], the update process consists of two steps. First, a fixed perception operator extracts local neighbourhood information using convolutional filters. Second, a shared neural network processes this local representation and produces a state increment that is added to the current state.

This formulation preserves the defining property of cellular automata - strict locality - while replacing the hand-crafted transition function with a learnable parametric model. Because the update rule is differentiable with respect to its parameters, the automaton can be trained by unrolling its dynamics over multiple steps and applying gradient descent to minimise a loss defined on the resulting configuration.

Neural cellular automata can therefore be understood as spatially distributed dynamical systems whose behaviour is shaped by optimisation rather than by manually specified rules.

3.4 Extension from Two to Three Dimensions

The original neural cellular automaton proposed by Mordvintsev et al. [13] operates on a two-dimensional grid. Extending this framework to three-dimensional voxel lattices is conceptually straightforward but introduces several practical and dynamical considerations.

In two dimensions, perception is typically performed using 3×3 convolutional filters applied to a Moore neighbourhood. In three dimensions, this naturally generalises to $3 \times 3 \times 3$ volumetric filters, increasing the neighbourhood size from nine to twenty-seven cells. As demonstrated in prior 3D extensions [20, 19, 9], Sobel-type gradient filters and other perception operators can be extended along the x, y, and z axes without altering the overall update formulation.

However, volumetric grids introduce important differences. The number of cells grows cubically with lattice resolution, which substantially increases memory requirements and computational cost. Moreover, since updates remain strictly local, information must propagate only through repeated applications of the update rule. In larger three-dimensional

grids, maintaining a coherent global structure therefore requires stable dynamics over a longer period of time.

These considerations suggest that behaviours observed in two-dimensional neural cellular automata - particularly persistence and regeneration - may not manifest identically in volumetric domains. While similar mechanisms can, in principle, be extended to three dimensions, the change in spatial topology and neighborhood structure can lead to differences in stability, information propagation, and emergent dynamics.

3.5 Core Components of a 3D Neural Cellular Automaton

The preceding sections introduced classical cellular automata, continuous extensions, and neural CA models. This section focuses on the principal theoretical components that form the foundation of a three-dimensional neural cellular automaton. The aim is to clarify the general concepts that govern these systems without delving into implementation-specific details or experimental results.

3.5.1 State Representation

In neural cellular automata, each voxel maintains a vector of real-valued channels. This notion originates from the two-dimensional model of Mordvintsev et al. [13], where multi-channel states support both visible pattern information and internal memory.

Channels can generally be divided into two categories:

- **Visible channels**, representing quantities directly compared to the target pattern, such as colour, occupancy, or density;
- **Hidden channels**, providing additional internal memory and facilitating information propagation between neighbouring cells.

If each cell contains C channels, its state can be represented as

$$\mathbf{s} \in \mathbb{R}^C.$$

Many neural CA models also include a continuous *alive* channel, which acts as a gating mechanism modulating whether updates are applied to a cell. This concept appears in the original NCA formulation [13] and has conceptual parallels in continuous CA such as SmoothLife [18] and Lenia [3], where real-valued states enable smooth evolution and gradient-based optimisation.

3.5.2 Perception Mechanism

Despite employing continuous state vectors, neural CA retain the defining property of locality: each cell updates based solely on information from its immediate neighbourhood [8, 23]. In neural CA, locality is realised by applying a fixed set of convolutional filters (e.g., 3×3 in 2D or $3 \times 3 \times 3$ in 3D) to extract spatial features [13, 20].

Typical filter types include:

- **Identity filters**, capturing a cell's own state;
- **Neighbour aggregation filters**, such as sums or averages of adjacent voxels;

- **Gradient filters**, including Sobel-type kernels extended to 3D, approximating directional differences and providing a local analogue of biological gradient sensing [13, 20].

The outputs of all applied filters are concatenated into a single perception vector for each cell, which is then passed to the update function. Using fixed, non-learnable filters stabilises training and maintains conceptual continuity between neural and classical cellular automata [13, 3].

3.5.3 Update Rule

The update rule is a shared, parametric function that governs how each cell's state evolves over time. Following the formulation of Mordvintsev et al. [13], the rule operates on the perceived neighbourhood features and produces a state increment added to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta\mathbf{s}.$$

This approach preserves the parallel, local nature of classical cellular automata while allowing the transition function to be learned through gradient-based optimisation. The update function is typically implemented as a lightweight neural network applied independently to each cell, such as a small multilayer perceptron or a sequence of 1×1 (or $1\times1\times1$ in 3D) convolutions [13]. These designs ensure that updates depend only on locally perceived information while keeping the parameter count sufficiently low to maintain stable training.

Two mechanisms commonly appear in neural CA formulations. Alive gating regulates updates based on the continuous alive channel, providing a soft analogue of classical alive/dead states and improving long-term stability [13]. Stochastic update masking applies updates probabilistically, preventing rigid synchronisation and encouraging robustness [13].

These components collectively support behaviours such as growth, pattern persistence, and regeneration, phenomena also observed in continuous morphogenetic systems such as Lenia [3]. Early three-dimensional neural CA models [19, 20, 9] demonstrate that the underlying update-rule principles generalise naturally to volumetric grids, although the emergent dynamics may differ from their 2D counterparts.

3.5.4 Grid Management

The neural CA is defined on a finite three-dimensional lattice, with boundary conditions chosen according to the experiment [13]. Key considerations include the lattice dimensions, initialization, and boundary treatment. Periodic or zero-value padding at the edges is often used to reduce boundary artefacts, though specific lattice sizes and boundary choices vary across implementations [16].

Three-dimensional grids scale cubically with lattice size, creating computational challenges. Research on large-scale volumetric simulations often employs data structures that reduce memory usage and accelerate neighbourhood access, such as Sparse Voxel Octrees [11] or the OpenVDB framework [14]. These approaches show how using locality-preserving representations can make simulations more scalable and offer guidance for extending neural CA to larger or more flexible volumetric setups.

3.5.5 Training and Optimisation

Because neural CA are differentiable, their parameters can be optimised using gradient-based methods to minimise a loss between the generated pattern and a target configura-

tion [13]. Training generally involves unrolling the CA dynamics over multiple steps and applying backpropagation through time.

Typical loss functions include mean squared error (MSE) for continuous targets and intersection-over-union or other domain-specific metrics for structural objectives [13, 16]. The choice of training regime—such as growth, persistence, or regeneration—affects the emergent behaviour and stability of the model. Optimisation often employs the Adam algorithm with learning rates around 0.001, sometimes supplemented by early stopping, data pooling, or logging frameworks [20].

These training strategies enable the discovery of update rules capable of producing robust self-organising structures in both two- and three-dimensional systems [19, 20, 9].

Chapter 4

Related Work

This chapter reviews prior work most directly related to neural cellular automata and their extensions. The focus is on implementations, training regimes, evaluation strategies, and attempts to extend these systems beyond two dimensions. Each section concludes by identifying limitations or open questions that motivate the three-dimensional framework developed in this thesis.

Some of the concepts discussed in this chapter were already introduced in Chapter 3. There, they were presented to establish the formal and conceptual background required for the definition of three-dimensional neural cellular automata. In contrast, the present chapter considers the same works from the perspective of prior research contributions, experimental results, and architectural design choices that serve as a direct basis for the approach developed in this thesis.

4.1 Growing Neural Cellular Automata

The development of neural cellular automata draws on a lineage of contributions spanning continuous self-organising systems, reaction-diffusion models of morphogenesis, and modern deep learning techniques for differentiable simulation. This section focuses on the seminal work of Mordvintsev, Randazzo, Niklasson, and Levin (2020) [13], which introduced the core paradigm of Growing Neural Cellular Automata and serves as the primary methodological foundation for the present thesis.

In their 2020 contribution, Mordvintsev et al. replaced the traditional discrete lookup-table transition rules of classical cellular automata with a small, shared neural network whose parameters are optimised end-to-end via gradient descent. The resulting model preserves the strict spatial locality of classical CA while enabling task-driven optimisation of the update dynamics, allowing the system to grow and persistently maintain predefined target patterns starting from simple initial conditions (often a single live cell or small seed).

Unlike earlier continuous or parameterised CA systems, which were often explored through manual parameter tuning or dynamical systems analysis, neural cellular automata emphasise supervised optimisation of update dynamics. Instead of searching for interesting rule sets, the transition function itself is trained to produce a predefined target pattern. This shift—from exploratory rule discovery to differentiable, task-driven optimisation—forms the methodological basis for the three-dimensional extension developed in this thesis.

While the core principles of neural cellular automata were introduced in Chapter 3, the focus here is on the concrete design and empirical properties of the 2020 implementation.

In particular, the architectural structure, stochastic update mechanism, and training procedure are analysed, as they define the core design principles underlying the three-dimensional extension developed in this thesis.

4.1.1 State Representation

Each cell in the original 2D model maintains a real-valued state vector of 16 channels. Four channels encode visible RGBA colour values compared directly against a target image. The remaining twelve channels serve as hidden internal memory.

The inclusion of hidden channels enables each cell to store and propagate information across time steps, effectively functioning as a distributed memory system embedded in the spatial lattice. Because updates are strictly local, long-range coordination must emerge from iterative short-range interactions. The hidden channels provide the representational capacity necessary for this coordination.

This architectural decision is central to the stability and regenerative behaviour observed in later experiments and directly informs the channel configurations adopted in subsequent work.

4.1.2 Perception Mechanism

Perception in the 2D neural CA is implemented through a fixed set of non-learnable convolutional filters applied to the full multi-channel grid. The standard configuration includes:

- An identity filter,
- Sobel filters approximating spatial derivatives in the x and y directions
- A Laplacian filter capturing local curvature-like information.

Applied across all 16 channels, these filters produce a 48-dimensional perception vector at each spatial location. A small multilayer perceptron operating independently at each cell transforms this perception vector into a state increment.

The use of fixed perception filters serves two purposes. First, it enforces locality and translation invariance. Second, it stabilises optimisation by constraining the form of spatial information available to the update rule. This design choice becomes particularly relevant in volumetric extensions, where filter dimensionality and neighbourhood size increase substantially.

4.1.3 Update Rule and Parameter Sharing

The update rule is implemented as a lightweight neural network shared across all cells. The output is a state increment added residually to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta\mathbf{s}.$$

Residual updates contribute to dynamical stability by encouraging incremental modifications rather than abrupt state transitions. Because the same parameters are applied at every spatial location, the learned rule defines a homogeneous dynamical system over the entire grid.

The complete architecture contains approximately 8000 trainable parameters, a scale small enough to emphasise that the complexity of the resulting structures arises from iterative local interaction rather than network size.

Stabilisation Mechanisms

Two mechanisms play a critical role in the robustness of the learned dynamics.

- **Alive gating** conditions state updates on whether a designated alpha channel exceeds a threshold. This introduces a soft analogue of classical alive/dead states and prevents inactive regions from destabilising active structures.
- **Stochastic update masking** applies updates to a randomly selected subset of cells at each time step. This prevents rigid synchronisation artefacts and encourages robustness to asynchronous perturbations.

These mechanisms collectively enhance long-term stability and regenerative capacity, properties that become increasingly significant when extending the model to larger or higher-dimensional grids.

4.1.4 Training Regimes

Mordvintsev et al. [13] introduced three principal training regimes: growing, persistent, and regenerating. Each of these regimes was designed to shape different aspects of the emergent dynamics. These regimes serve as the direct methodological foundation for the training strategies explored in the three-dimensional extension developed in this thesis. Rather than relying solely on exploratory parameter search as in earlier continuous CA systems, neural cellular automata exploit gradient-based optimisation to discover update rules tailored to specific behavioural objectives, with the choice of regime determining the balance between initial pattern formation, long-term stability, and robustness to perturbation.

Growing Regime

In the growing regime, the model is trained to produce a target image from a single active seed. The loss is computed after a fixed number of update steps. The model learns to expand outward and approximate the target pattern.

Once the target is reached, no additional constraint is imposed. As a result, long-term stability is not guaranteed. Patterns may drift, decay, or diverge if simulation continues beyond the training horizon.

Persistent Regime

The persistent regime extends training beyond initial convergence. The loss is evaluated at multiple time steps after the target is reached, encouraging the system to maintain the pattern over extended simulations.

This regime induces greater dynamical stability. Interestingly, persistent models exhibit limited regenerative behaviour even without explicit damage training, suggesting that stability mechanisms learned for persistence partially generalise to perturbation recovery.

Regenerating Regime

In the regenerating regime, random damage is introduced during training by zeroing out contiguous regions of the grid. The model must reconstruct the target pattern from the damaged configuration.

This explicit training objective leads to robust regenerative dynamics. Regenerating models can recover from substantial perturbations and maintain structural integrity over long simulation horizons.

4.1.5 Relevance to the Present Thesis

The present thesis adopts the architectural principles, training regimes, and optimisation strategy introduced by Mordvintsev et al. [13] as its baseline. The central contribution lies in extending this framework to three-dimensional voxel lattices and systematically examining how architectural adaptation and training strategy interact with volumetric spatial structure.

In particular, the transition from 2D to 3D increases neighbourhood size, computational cost, and information propagation requirements. Whether the stability and regenerative behaviours observed in planar domains persist under these conditions is not addressed in the original work and forms a central motivation for the experiments conducted here.

Chapter 5

Implementation

TODO

Chapter 6

Testing

TODO

Chapter 7

Experiments

TODO

Chapter 8

Conclusion

TODO

Bibliography

[1] Berlekamp, E. R.; Conway, J. H. and Guy, R. K. *Winning Ways for Your Mathematical Plays, Volume 2*. Academic Press, 1982.

[2] Béna, G.; Faldor, M.; Goodman, D. F. M. and Cully, A. *A Path to Univrsal Neural Cellular Automata*. 2025. Available at: https://arxiv.org/abs/2505.13058.

[3] Chan, B. W.-C. *Lenia: Biology of Artificial Life*. 2019.

[4] Community, P. *PyTorch Documentation*. 2025. Available at: https://docs.pytorch.org/stable/index.html.

[5] Cook, M. *Universality in Elementary Cellular Automata*. 2004.

[6] Gardner, M. *Mathematical Games: The fantastic combinations of John Conway's new solitaire game "Life"*. 1970. Available at: https://web.stanford.edu/class/sts145/Library/life.pdf.

[7] Goodfellow, I.; Bengio, Y. and Courville, A. *Deep Learning*. MIT Press, 2016. Available at: https://www.deeplearningbook.org.

[8] Ilachinski, A. *Cellular Automata: A Discrete Universe*. World Scientific, 2001. ISBN 981-02-4623-4.

[9] Jha, A. *3D Growing Neural Cellular Automata*. GitHub, 2024. Available at: https://github.com/Aadityaza/3d-Growing-neural-cellular-automata. Originally published under username Aadityaza.

[10] Kinderen, E. de and Wuensche, A. 3D CA in a fluid dynamics PDE. In: *Proceedings of the Second International Conference on Evolvable Systems: From Biology to Hardware*. Springer, 1998.

[11] Laine, S. and Karras, T. Efficient sparse voxel octrees. In: *Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. New York, NY, USA: Association for Computing Machinery, 2010. ISBN 9781605589398. Available at: https://doi.org/10.1145/1730804.1730814.

[12] Marinescu, D. C. Chapter 2 - Nature-Inspired Algorithms and Systems. In: Marinescu, D. C., ed. *Complex Systems and Clouds*. Boston: Elsevier, 2017, p. 33–63. Computer Science Reviews and Trends. ISBN 978-0-12-804041-6. Available at: https://www.sciencedirect.com/science/article/pii/B9780128040416000025.

[13] Mordvintsev, A.; Randazzo, E.; Niklasson, E. and Levin, M. *Growing Neural Cellular Automata*. 2020. Available at: https://distill.pub/2020/growing-ca/.

[14] Museth, K. VDB: High-resolution sparse volumes with dynamic topology. *ACM Trans. Graph.* New York, NY, USA: Association for Computing Machinery, 2013. ISSN 0730-0301. Available at: https://doi.org/10.1145/2487228.2487235.

[15] Neumann, J. von. *Theory of Self-Reproducing Automata.* University of Illinois Press, 1966.

[16] Niklasson, E.; Mordvintsev, A.; Randazzo, E. and Levin, M. Self-Organising Textures. *Distill*, 2021. Https://distill.pub/selforg/2021/textures/.

[17] Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* 2019. Available at: https://arxiv.org/abs/1912.01703.

[18] Rafler, S. *Generalization of Conway's "Game of Life" to a continuous domain – SmoothLife* arXiv preprint arXiv:1111.1567. 2011. Available at: https://doi.org/10.48550/arXiv.1111.1567.

[19] Rossi, F. and Orsini, M. *3D Growing Neural Cellular Automaton.* GitHub, 2021. Available at: https://github.com/SkyLionx/3d-cellular-automaton. Originally published under username SkyLionx.

[20] Sudhakaran, S.; Grbic, D.; Li, S.; Katona, A.; Najarro, E. et al. *Growing 3D Artefacts and Functional Machines with Neural Cellular Automata.* July 2021, ALIFE 2021: The 2021 Conference on Artificial Life, p. 108. ALIFE 2022: The 2022 Conference on Artificial Life. Available at: https://doi.org/10.1162/isal_a_00451.

[21] Tobias, P. A. G. *3D Cellular Automata.* 2009. Available at: https://eprints.bournemouth.ac.uk/17408/.

[22] Wolfram, S. *Statistical Mechanics of Cellular Automata.* 1983.

[23] Wolfram, S. *A New Kind of Science.* Wolfram Media, 2002. ISBN 1-57955-008-8. Available at: https://www.wolframscience.com/nks/.